

[CS230] Optimization Algorithms: Hyperparameter Tuning Strategy

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-4. Neural Networks Basics - *Completed*

Chapter 5. Practical Aspects of Deep Learning (Current Unit)

- 5.1-5.5 Regularization & Data Setup - *Completed*
- 5.6-5.9 Optimization (Mini-batch, Adam, Decay) - *Completed*
- **5.10 Hyperparameter Tuning Strategy**
 - * Tuning Priority (Alpha is King)
 - * Grid Search vs Random Search
 - * Picking Appropriate Scale (Log Scale)
 - * Coarse to Fine Strategy
- 5.11 Batch Normalization - *Upcoming*

지난 시간 복습 및 연결

우리는 지금까지 신경망을 만들고(Architecture), 학습시키고(Adam), 규제(Regularization) 하는 모든 방법을 배웠습니다. 하지만 막상 여러분이 모델을 돌리려고 하면 거대한 벽에 부딪힙니다. "학습률은 0.01? 0.0001?", "배치 크기는 32? 64?" 수십 개의 다이얼을 무작위로 돌리는 것은 도박입니다. 우리는 체계적이고 과학적인 탐색 전략이 필요합니다.

1 Unit Overview

□ 핵심 목표

이 단원은 SOTA(State-of-the-art) 성능을 달성하기 위한 하이퍼파라미터 튜닝의 우선순위와 탐색 기법을 다룹니다.

- **우선순위:** 학습률(α)이 가장 중요하다는 계층 구조를 이해합니다.
- **전략:** 고차원 공간에서는 **Random Search**가 Grid Search보다 압도적으로 유리한 이유를 기하학적으로 증명합니다.
- **스케일:** 학습률 등을 탐색할 때 선형(Linear)이 아닌 로그 스케일(Log Scale)을 써야 하는 수학적 이유를 배웁니다.

2 Essential Terminology

기법	설명	비유
Grid Search	모든 조합을 격자무늬로 다 해보는 것.	자물쇠 번호를 0000부터 9999까지 다 돌
Random Search	무작위로 값을 찍어보는 것.	감으로 찍어서 맞춘 (고차원에서 유리).
Log Scale	자릿수 단위로 탐색 (10^{-4} , 10^{-3} ...).	현미경 배율을 10배, 100배로 조절하며
Coarse to Fine	넓게 훑고 (Coarse), 좋은 곳을 집중 공략 (Fine).	숲 전체를 스캔하고, 의심 가는 구역만 수

3 Core Concepts: 무엇이 중요한가?

3.1 1. Tuning Priority (우선순위 계급도)

모든 파라미터가 평등하지 않습니다. 앤드류 응 교수의 경험적 가이드라인입니다.

- **Tier 1 (King - 가장 중요):**
 - **Learning Rate (α):** 이것이 틀리면 다른 걸 아무리 잘 맞춰도 소용없습니다.
- **Tier 2 (Queen - 중요):**
 - Momentum (β), Mini-batch Size, Hidden Units 개수.
- **Tier 3 (Pawn - 덜 중요):**
 - Layer 개수, Learning Rate Decay.
- **Do Not Touch (건드리지 마세요):**
 - Adam의 $\beta_1(0.9)$, $\beta_2(0.999)$, $\epsilon(10^{-8})$.

3.2 2. Grid Search vs Random Search

□ 보물 찾기 (직관적 비유)

지도상에 보물이 어디 있는지 모릅니다.

- **Grid Search:** 지도를 바둑판처럼 나누고 교차점만 팝니다. 만약 보물이 교차점 사이에 있다면? 영원히 못 찾습니다.
- **Random Search:** 지도를 무작위로 콕 찍습니다. 같은 횟수를 시도하더라도, 중요한 파라미터에 대해 훨씬 더 다양한 값을 테스트해볼 수 있습니다.

3.3 3. Scale Matters: Log Scale Sampling

학습률 α 를 0.0001에서 1 사이에서 찾는다고 합시다.

나쁜 방법 (Linear): `np.random.rand()`

- 90%의 값이 0.1 ~ 1 구간에 몰립니다.

- 정작 중요한 $0.0001 \sim 0.1$ 구간은 전체의 10%밖에 탐색하지 못합니다.

좋은 방법 (Log Scale):

- $10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}$ 각 구간을 공평하게 탐색해야 합니다.
- 지수(r)를 $-4 \sim 0$ 사이에서 뽑고, 10^r 을 계산합니다.

4 Implementation: Scientific Search

올바른 스케일로 파라미터를 뽑는 함수를 구현합니다.

```

1 import numpy as np
2
3 class HyperparameterSearch:
4     def sample_linear(self, low, high, num_samples):
5         """
6         은닉 유닛 수, 층 수 등 등간격이( 의미 있는 경우)
7         """
8         return np.random.uniform(low, high, num_samples)
9
10    def sample_log_scale(self, low_exp, high_exp, num_samples):
11        """
12        학습률(alpha), 정규화상수(lambda) 등 자릿수가( 중요한 경우)
13        범위: 10^low_exp ~ 10^high_exp
14        """
15        # 1. 지수(r)를 균등하게 뽑음 (-4 ~ 0)
16        r = np.random.uniform(low_exp, high_exp, num_samples)
17        # 2. 의10 거듭제곱으로 변환
18        return 10 ** r
19
20    def sample_beta(self, num_samples):
21        """
22        Momentum Beta (0.9 ~ 0.999)
23        1-beta 값을 로그 스케일로 뽑는 것이 핵심!
24        """
25        # 1-beta 범위: 0.1 ~ 0.001 (10^-1 ~ 10^-3)
26        r = np.random.uniform(-3, -1, num_samples)
27        return 1 - (10 ** r)
28
29 # --- 실행 ---
30 if __name__ == "__main__":
31     searcher = HyperparameterSearch()
32
33     # 학습률 탐색: 0.0001 ~ 1.0
34     alphas = searcher.sample_log_scale(-4, 0, 5)
35     print("Alphas:", np.round(alphas, 6))
36
37     # 모멘텀 탐색: 0.9 ~ 0.999
38     betas = searcher.sample_beta(5)
39     print("Betas:", np.round(betas, 6))

```

Listing 1: Hyperparameter Sampling Strategies

5 FAQ & Pitfalls

□ Coarse to Fine 전략 (실전 팁)

처음부터 100 Epoch씩 돌리며 완벽한 값을 찾으려 하지 마세요. 1. **Coarse**: 넓은 범위에 서 5 10 Epoch만 짧게 돌려 대략적인 성능을 봅니다. 2. **Zoom In**: 성능이 좋은 영역을 발견하면 그 구간을 집중 확대합니다. 3. **Fine**: 좁은 영역에서 정밀하게 다시 Random Search를 수행합니다.

Q. β (Momentum)는 왜 $1 - \beta$ 로 로그 샘플링하나요?

A. β 는 1에 가까워질수록 민감해지기 때문입니다. $0.9 \rightarrow 0.9005$ 는 별 차이 없지만, $0.999 \rightarrow 0.9995$ 는 평균 기간이 1000일에서 2000일로 2배가 됩니다. 1에 가까운 값을 더 세밀하게 탐색해야 합니다.

다음 단계 (Next Step)

이제 최적의 파라미터까지 찾았습니다. 하지만 모델이 깊어질수록 "학습 속도가 느려지고 초기값에 너무 민감해지는 문제"가 발생합니다. 데이터 분포가 층을 지날 때마다 틀어지기 때문입니다.

이를 해결하기 위해 딥러닝 역사상 가장 위대한 발명 중 하나인 [Batch Normalization]이 등장했습니다. 다음 시간에 이 마법 같은 기법을 파헤쳐 보겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Priority**: 학습률(α)이 1순위다.
2. **Strategy**: Grid Search보다는 **Random Search**가 효율적이다.
3. **Log Scale**: α 나 λ 는 자릿수 단위로 탐색해야 한다. (10^{-4} , 10^{-3} ...)
4. **Process**: 넓게 훑고(Coarse), 좁게 파고들어가라(Fine).